

Interface & Data Contracts (SCRUM-182 / Backlog 1.4)

Status: In Progress · Draft for team review **Owner:** Bilal · **Sprint 1**

Source of truth for: SCRUM-153 (Pydantic schemas), SCRUM-156 (SQLite schema), SCRUM-152 (API scaffold).

If code and this document disagree, fix one of them in the same PR.

1. Scope and conventions

MVP vs Stretch : the MVP input is a curated image set that the team collects and uploads (publicly released Capital One and other brand photography). The Web Crawler and URL submission are stretch goals (Reach). They are kept in this document as reserved contracts, marked [STRETCH], so adding them later needs no redesign, but nothing in the MVP depends on them.

This document defines what each component receives and returns, using the components in the Design Document. The UI talks only to the Backend API; the Backend API is the only caller of every other component and the only reader/writer of Data Storage.

- Transport (UI ↔ API): HTTPS, JSON, except image upload (`multipart/form-data`). Base path `/api/v1`.
- Identifiers: server-generated UUID4 strings (`job_id`, `image_id`, `feedback_id`).
- Timestamps: ISO 8601 UTC (`2026-10-05T14:03:22Z`).
- Scores: criterion scores and overall score are **0–100** (float). Confidence is **0–1**.
- Criteria are referenced by `criterion_id` (string slug). The list of criteria comes from the rubric JSON config (SCRUM-180 will finalize the 4–6 MVP criteria). **No criterion name is hard-coded anywhere in this document or in code.**
- Every error uses one envelope (section 5).

MVP criteria

criterion_id* (proposed slug)	Criterion	Produced by
exposure_brightness	Exposure and brightness	CV measurement
contrast	Contrast	CV measurement
color_palette_fit	Color palette fit (vs. configurable brand palette)	CV measurement
framing_subject_position	Framing and subject position	CV measurement
sharpness_subject_emphasis	Sharpness and subject emphasis	CV measurement
tone_approachability	Tone and approachability (tone label,	VLM, the only model judgment

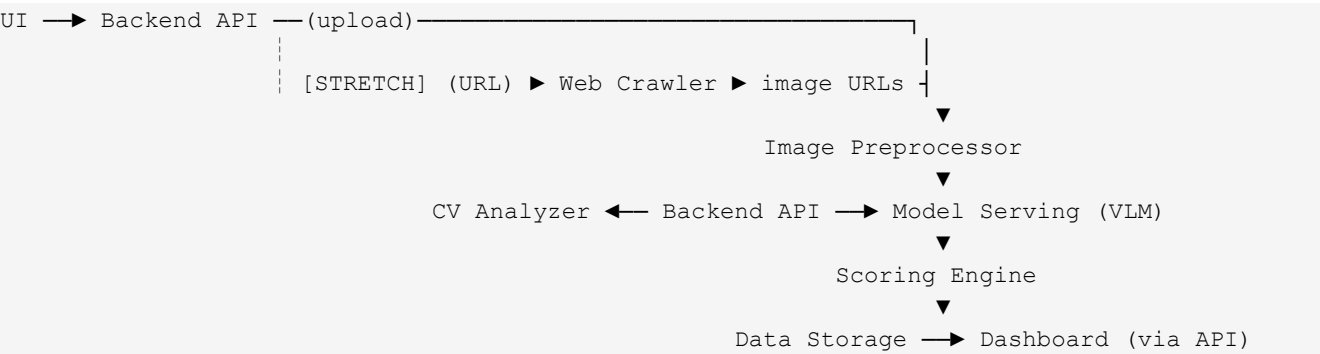
	score, one evidence sentence)	
--	-------------------------------	--

]

First alternates if the mentor swaps one: `background_simplicity` (CV), `authenticity` (VLM). Slugs and contents live in `brand_standards.json`; if Tuesday changes the set, only that file changes, not the API contract.

2. Data flow

MVP path (solid) is upload only. The crawler branch (dashed) is **[STRETCH]**.



The Backend API runs these stages inside one background job per submission, so the UI gets a `job_id` immediately and polls for progress.

3. UI ↔ Backend API (public REST contract)

#	Method & path	Purpose	Request	Success response
1	GET /health	Liveness check (SCRUM-152)	none	200 {"status":"ok","version":"0.1.0"}
2	GET /api/v1/rubrics	List selectable rubrics/criteria for Screen 1	none	200 RubricList
3	POST /api/v1/analyses/uploads	Submit photos	multipart: files[] (1..N), rubric_id, criterion_ids[] (optional subset)	202 JobAccepted
4	POST /api/v1/analyses/urls [STRETCH]	Submit website URL (schema reserved; MVP returns 501 NOT_IMPLEMENTED)	JSON UriSubmission	202 JobAccepted

5	GET /api/v1/jobs/{job_id}	Poll status/progress (Screen 2)	none	200 JobStatus
6	GET /api/v1/jobs/{job_id}/results	Dashboard data (Screen 3)	none	200 JobResults (409 if job not finished)
7	POST /api/v1/images/{image_id}/feedback	Agree/Disagree on AI reasoning	JSON FeedbackSubmission	201 FeedbackRecord

3.1 Payload shapes

```
// UrlSubmission (request, #4)
{ "url": "https://www.capitalone.com/", "rubric_id": "default-v1",
  "criterion_ids": [...],           // optional; omitted = all criteria in rubric
  "max_pages": 10 }                // optional, server caps it

// JobAccepted (response, #3 and #4)
{ "job_id": "uuid", "status": "queued", "input_type": "upload|url",
  "created_at": "...", "status_url": "/api/v1/jobs/{job_id}" }

// JobStatus (response, #5)
{ "job_id": "uuid",
  "status":
    "queued|crawling|preprocessing|analyzing|scoring|completed|completed_with_errors|failed",
    // "crawling" is used only by the [STRETCH] URL path
  "progress": { "images_total": 12, "images_done": 5, "images_failed": 1 },
  "created_at": "...", "updated_at": "...",
  "error": null }                  // ErrorBody only when status == failed

// JobResults (response, #6)
{ "job_id": "uuid", "rubric_id": "default-v1",
  "summary": { "images_scored": 11, "mean_overall": 72.4,
    "score_distribution": [{"bucket": "0-20", "count": 0}, "..."],
    "flagged_count": 3 },
  "images": [ ImageResult, ... ] }

// ImageResult
{ "image_id": "uuid", "source": "upload:filename.jpg | https://site/img.jpg",
  "status": "scored|partial|failed|filtered_out",
  "overall_score": 71.0,           // null if failed/filtered_out
  "flagged": false,
  "measurements": { ... },         // object defined in section 4.3 (provisional keys)
  "criteria": [ CriterionResult, ... ],
  "feedback": null }              // latest reviewer feedback if any

// CriterionResult
{ "criterion_id": "string", "score": 80.0,           // null if unavailable
  "weight": 0.25,
  "source": "cv|vlm",
  "label": null,                       // tone label for the VLM criterion, else null
  "explanation": "Human-readable reasoning...",
  "evidence": ["short observation", "..."],
```

```

"availability": "ok|unavailable_needs_review" }

// FeedbackSubmission (request, #7)
{ "verdict": "agree|disagree", "comment": "optional, <= 1000 chars",
  "criterion_id": null } // null = whole-image feedback

```

Decision on status values: the job has 8 states (above). An image has 4: `scored`, `partial` (qualitative criteria unavailable, CV-only score), `failed`, `filtered_out` (non-photographic, removed by preprocessor). `completed_with_errors` means at least one image was `partial` or `failed` but the batch finished. This implements the failure path from the Designing-a-Solution workshop: one bad model call never fails the batch.

4. Internal component contracts (all called by the Backend API)

4.1 Web Crawler [STRETCH] (URL → image URLs)

```

// in
{ "url": "https://...", "max_pages": 10, "same_domain_only": true, "timeout_s": 30 }
// out
{ "pages_visited": 7,
  "images": [ { "image_url": "https://...", "page_url": "https://...",
    "alt_text": "optional", "width_hint": 1200, "height_hint": 800 } ] }

```

Not built in the MVP (Reach: SCRUM-178, 179, 217). Owns traversal and asset discovery only; downloading the discovered URLs is a separate async downloader (SCRUM-179) that writes to `cache/raw/` and then hands off like an upload. In the MVP, the team's curated image set enters through upload (#3) instead. Failure: unreachable URL → `CRAWL_FAILED` (job failed); a single page failing is skipped and counted.

4.2 Image Preprocessor (file on disk → standardized photo or rejection)

Already implemented in `src/brandlens/preprocessing/` (SCRUM-117 and 118, Done). The contract wraps those functions; the integration into one step is SCRUM-128 (Sanish).

What the existing code does:

- `load_image(path, supported_formats=("JPEG", "PNG", "WEBP"), max_pixels=25_000_000)` detects format from file contents and raises on unsupported, animated, oversized, or corrupt files.
- `filter_photo(image, FilterSettings(min_short_side=64, max_aspect_ratio=5.0))` returns the image or `None`. It returns no reason, so the integration step must work out the reason (it can recompute from the image size).
- `standardize_image(image, StandardizationSettings(max_size=None, background=(255,255,255)))` fixes EXIF orientation, converts to RGB/sRGB (transparency composited onto the background), never crops or upscales, and rejects CMYK without an ICC profile.

```

// in
{ "image_id": "uuid", "raw_path": "cache/raw/<id>", "source": "upload:name.jpg" }
// out
{ "image_id": "uuid", "decision": "keep|reject",

```

```

"reject_reason":
"unsupported_format|too_many_pixels|animated|too_small|extreme_aspect_ratio|unprocessable
|null",
"processed_path": "cache/processed/<id>.png",
"width": 1024, "height": 683, "color_mode": "RGB" }

```

Rejected images are stored as `filtered_out` with the reason, not silently dropped. `unprocessable` covers color-profile failures. Settings (`min_short_side`, `max_aspect_ratio`, `max_size`) are preprocessor configuration, not API fields. **Note:** the 25 MP pixel limit is stricter than a file-size limit and should be matched by the upload endpoint's limits (section 5).

4.3 Computer Vision Analyzer (`processed image → measurements`)

In: `processed_path` plus the rubric's brand palette. Out: one `measurements` object. Pure computation, deterministic for a given image. **All keys are provisional** until the CV spike (SCRUM-116) and the measurement tickets (129, 130, 131) land; adding keys is additive and safe, renaming keys needs a contract update.

Criterion	Measurements (proposed keys)	Jira
exposure_brightness	mean_luminance (0-255), near_black_fraction, near_white_fraction	SCRUM-129
contrast	luminance_std, p5_p95_range	SCRUM-129
color_palette_fit	dominant_colors: [{hex, proportion}], palette_distance (to rubric palette)	SCRUM-130 (k-means); distance step has no ticket
framing_subject_position	subject_bbox (normalized x,y,w,h), center_offset, thirds_offset, edge_margin_min	SCRUM-131
sharpness_subject_emphasis	subject_sharpness, background_sharpness, sharpness_ratio	no ticket

The literature review notes that framing and sharpness both need a subject region first (e.g. a saliency or center-weighted estimate). That is one shared step inside the analyzer; whoever owns SCRUM-131 should also expose it for sharpness.

4.4 Model Serving / VLM (`image + tone criterion → one tone judgment`)

The model makes exactly one judgment in the MVP: tone and approachability. Transport: HTTP to a vLLM/Ollama OpenAI-compatible `/v1/chat/completions` endpoint, so the model can be swapped without changing other components. The Backend's model client enforces this contract with Pydantic (SCRUM-137, 138):

```

// in (what our client builds; only selected VLM criteria are sent, SCRUM-135)
{ "image_id": "uuid",
  "criteria": [ { "criterion_id": "tone_approachability", "name": "Tone and
approachability",

```

```

        "definition": "...", "evaluation_evidence": "...",
        "allowed_labels": ["approachable", "confident"] } ] }
// out (must be valid JSON matching this, else treated as a model failure)
{ "judgments": [ { "criterion_id": "tone_approachability",
        "label": "approachable",          // one of allowed_labels
        "fit_score": 4,                    // integer 1-5
        "evidence": "One sentence naming something visible in the photo." } ] }

```

Invalid output examples: label not in `allowed_labels`, `fit_score` outside 1-5, empty evidence, or more than one sentence of evidence (soft rule). The generic list shape is kept so a second VLM criterion (e.g. authenticity) can be added without a contract change. Development runs on local Ollama (SCRUM-120), production on vLLM (SCRUM-136).

Failure handling: timeout, HTTP error, or invalid JSON triggers one retry, then the image is marked `partial`, the tone criterion becomes `unavailable_needs_review`, and the other five CV criteria still score (SCRUM-139).

4.5 Scoring Engine (`measurements + judgments + rubric → scores`)

In: `measurements`, `judgments` (may be empty), rubric from `brand_standards.json` (section 4.7). Out: `overall_score`, `criteria[]` (`CriterionResult`), `flagged`. Pure function, no I/O (SCRUM-121, 122, 123).

- CV criteria: each criterion's `params` in the rubric map measurements to a 0-100 score (e.g. target brightness range).
- Tone: `fit_score` 1-5 maps to 0-100 via $(\text{fit_score} - 1) / 4 * 100$; the label and evidence sentence become `label` and `explanation`.
- Weights are renormalized over available criteria; an image is `flagged` if overall is below the rubric `flag_threshold` or any criterion is unavailable.

4.6 Data Storage (Backend API only)

Persists: `batches/jobs`, `images`, `criterion_scores` (with explanations), `feedback`, and cached files under `cache/`. The table design is in SCRUM-156; the mapping from the payloads above is:

Payload	Stored as
JobAccepted/JobStatus	jobs row
ImageResult (header, measurements)	images row
CriterionResult	criterion_scores row (FK image)
FeedbackSubmission	feedback row (FK image, optional criterion)
Rubric config	JSON file on disk, rubric_id recorded on the job

4.7 Rubric configuration (`brand_standards.json`)

Owned by SCRUM-181 (Sanish), content from SCRUM-180 (Krishna). Jira requires each criterion to have a

name, definition, and intended evaluation evidence. Proposed shape for SCRUM-181 to adopt or amend:

```
{ "rubric_id": "default-v1", "name": "Capital One MVP photography rubric",
  "flag_threshold": 60,
  "brand_palette": ["#004977", "#D03027", "#FFFFFF"],           // placeholder hex values,
  Capital One to confirm
  "allowed_tones": ["approachable", "confident"],
  "criteria": [
    { "criterion_id": "exposure_brightness", "name": "Exposure and brightness",
      "definition": "...", "evaluation_evidence": "...",
      "method": "cv", "weight": 0.2, "params": { } },
    // ... contrast, color_palette_fit, framing_subject_position,
    //   sharpness_subject_emphasis (all method "cv")
    { "criterion_id": "tone_approachability", "name": "Tone and approachability",
      "definition": "...", "evaluation_evidence": "...",
      "method": "vlm", "weight": 0.2 } ] }                      // weights sum to 1.0
```

GET /api/v1/rubrics returns this shape (without internal-only fields), so the UI criteria picker and the scoring engine read one source. The palette values above are placeholders, not real brand colors.

5. Error envelope and codes

```
{ "error": { "code": "VALIDATION_ERROR", "message": "Human readable", "details": [] } }
```

HTTP	code	When
422	VALIDATION_ERROR	Payload fails Pydantic validation (bad URL, empty files, bad verdict)
415	UNSUPPORTED_FILE_TYPE	Upload is not JPEG/PNG/WebP
413	FILE_TOO_LARGE	Over per-file limit (proposed 10 MB)
404	NOT_FOUND	Unknown job_id / image_id / rubric_id
409	JOB_NOT_FINISHED	Results requested while job is running
501	NOT_IMPLEMENTED	Stretch endpoint called before it exists (URL submission)
500	INTERNAL_ERROR	Anything unexpected (no stack traces to client)

6. Open items and assumptions (need team confirmation)

- 1. **Criteria are final only after Tuesday's mentor review.** Krishna's email proposes six; SCRUM-180 is In Review. Contracts are criterion-agnostic; only brand_standards.json changes.

2. **Ticket gaps found:** no ticket for sharpness and subject-emphasis measurement, none for palette-distance scoring, and none for the shared subject-region step (framing and sharpness both need it). Raise with Sanish and Krishna.
3. **Rubric field list (4.7)** needs Sanish's agreement before SCRUM-181 builds the file; `brand_palette` hex values must come from Capital One.
4. **Model choice** is unresolved (benchmarking is SCRUM-119, not started). Contract 4.4 is model-neutral.
5. **Wireframes (SCRUM-111, In Progress)** should be checked against the `JobStatus` and `JobResults` payloads, especially Screen 3.
6. **Assumptions to confirm:** 0-100 scores, flag threshold in the rubric, 10 MB per file plus the 25 MP pixel limit from the preprocessor, polling instead of websockets.
7. **Crawler and URL endpoint are stretch** (SCRUM-178, 179). The schema is reserved; the endpoint returns `501` in the MVP. If built, crawler output goes through the Backend API per the Design Document.
8. **Evaluation images:** the team still needs to collect a public-only set (no internal Capital One data). Keep it out of the API contract (e.g. `data/eval_set/`).
9. **Repo and docs mismatches to fix:** the README and Design Document still describe the crawler as core; `pyproject.toml` and `LICENSE.txt` say GPL-3.0 while the Design Document says Apache 2.0.